Lecture 27.03: Handling Missing Values (Statistical Imputation)

📌 Topic Overview

In this lecture, Shraddha ma'am inspects missing values across the loan application dataset and applies Scikit-Learn's **SimpleImputer** to impute numerical and categorical columns without dropping rows.

🏦 Imputation Strategy



💻 Python Implementation

```
import pandas as pd
from sklearn.impute import SimpleImputer

def impute_loan_missing_values(df):
    """
    Imputes missing values using statistical strategies:
    - Numerical features (Income, DTI, Credit_Score): Mean or Median strategy
    - Categorical features (Employment, Marital_Status): Most Frequent (Mode) stra
    """
    df_clean = df.copy()

    # Identify numerical and categorical columns with missing values
    num_cols = df_clean.select_dtypes(include=['int64', 'float64']).columns
    cat_cols = df_clean.select_dtypes(include=['object']).columns

    # 1. Numerical Imputation (Median is robust against extreme outliers)
    num_imputer = SimpleImputer(strategy='median')
    df_clean[num_cols] = num_imputer.fit_transform(df_clean[num_cols])
```

```
    # 2. Categorical Imputation (Most frequent / Mode)
    cat_imputer = SimpleImputer(strategy='most_frequent')
    df_clean[cat_cols] = cat_imputer.fit_transform(df_clean[cat_cols])

    missing_remaining = df_clean.isnull().sum().sum()
    return df_clean, missing_remaining

# --- Function Call & Testing ---
# clean_df, nulls_left = impute_loan_missing_values(raw_df)
# print("Remaining Missing Values after Imputation:", nulls_left)
```

🔑 Key Takeaways

1. Never blindly drop missing rows (`dropna()`); throwing away 20% of customer applications loses valuable signal.
2. Use **Median** for skewed numerical features like Income, and **Most Frequent (Mode)** for categorical labels.